# Optimization in Data Analysis

Project 5.3: Minimization of
Weakly Convex Functions


| Marta Szuwarska | Mateusz Nizwantowski | Łukasz Grabarski |
|---|---|---|
| 01171269@pw.edu.pl | 01161932@pw.edu.pl | 01171233@pw.edu.pl |
| 320662 | 313839 | 320627 |


June 2026

# Contents

**1 Introduction** **3**

**2 Mathematical background** **3**
2.1 What kind of problem are we solving? . . . . . 3
2.2 The algorithm idea . . . . . 3
2.3 The Moreau envelope . . . . . 4

**3 The main algorithm** **4**
3.1 The generic solver . . . . . 4
3.2 The three update formulas . . . . . 5
3.3 Computing the progress measure . . . . . 5

**4 Code structure** **5**

**5 Experiments** **6**
5.1 Reproduction - Phase Retrieval . . . . . 6
5.2 Reproduction - Blind Deconvolution . . . . . 7
5.3 Comparison with Gradient-Based Optimizers . . . . . 8
5.4 Moreau Envelope Gradient Norm . . . . . 10
5.5 Stopping Criteria . . . . . 12
5.6 Convergence Rate Experimentally . . . . . 12
5.7 Empirical Recovery Probability . . . . . 13

**6 Summary** **14**

# 1 Introduction

A lot of useful problems in signal processing and machine learning are *hard* in two ways at once: the function we want to minimize is not smooth (it has kinks), and it is not convex (it has more than one local minimum). Examples include phase retrieval, blind deconvolution, and robust regression. For functions like this, the usual theory behind gradient descent does not give any guarantee, so for a long time, people used simple iterative methods without really knowing why they worked.

The paper we are reproducing, Davis and Drusvyatskiy (2019), finally gives the guarantee we were missing before. The key observation is that even though these functions are hard to work with, they might have one redeeming feature called **weak convexity**: if you add a small quadratic bump $\frac{\rho}{2}\|x\|^2$ to them, they become convex. The authors then analyze a whole family of methods - which includes the stochastic subgradient, prox-linear, and proximal-point methods - using a tool called the **Moreau envelope**. Their main result is a clean convergence rate: a sensible measure of "how close we are to a stationary point" decreases at a rate of $O(k^{-1/4})$.

Our project implements this framework in PyTorch and tests it. This document describes the math we need (Section 2), the main functions in our code (Sections 3 and 4), and the experiments with all the figures we generated (Section 5).

# 2 Mathematical background

## 2.1 What kind of problem are we solving?

We want to solve problems of the form:

$$\min_{x\in\mathbb{R}^d} \varphi(x) \;=\; \underbrace{\mathbb{E}_\xi\big[f(x,\xi)\big]}_{\text{average loss over data}} \;+\; \underbrace{r(x)}_{\text{regularizer}} \quad . \tag{1}$$

Here every $f(\cdot,\xi)$ is $\rho$-weakly convex and $r$ is a simple convex regularizer (it can be nonsmooth, like the $\ell_1$ norm). "$\rho$-weakly convex" means: $x \mapsto \varphi(x) + \frac{\rho}{2}\|x\|^2$ is convex. In other words, the function is allowed to curve downward, but not too sharply - the amount of allowed downward curvature is controlled by $\rho$.

Our running example is **phase retrieval**. We are given measurement vectors $a_i$ and numbers $b_i = \langle a_i, \bar{x}\rangle^2$, and we want to recover the hidden signal $\bar{x}$. The per-sample loss is

$$f(x,\xi) = \big|\, \langle a, x\rangle^2 - b \,\big|,$$

which is $2\|a\|^2$-weakly convex. The absolute value gives the kinks, and the square gives the nonconvexity - exactly the hard combination above.

## 2.2 The algorithm idea

At each step we pick a random data point, replace $f$ by a simple **model** $f_{x_t}$ of it around the current point $x_t$, and then take a *proximal step*: move to the point that minimizes the model plus a penalty for moving too far,

$$x_{t+1} \;=\; \arg\min_y \Big\{ \underbrace{f_{x_t}(y,\xi_t)}_{\text{model}} + r(y) + \underbrace{\tfrac{\beta_t}{2}\|y - x_t\|^2}_{\text{don't move too far}} \Big\}. \tag{2}$$

The number $\beta_t$ is the inverse step-size: large $\beta_t$ means small, cautious steps. The only thing that changes between the three methods is how honest the model is:

| Method | Model $f_x(y,\xi)$ | Idea |
|---|---|---|
| Subgradient | $f(x)+\langle v, y-x\rangle$ | linear approximation (a tangent line) |
| Prox-linear | linearize only the *inside* of $\lvert\cdot\rvert$ | keeps the kink, drops the curvature |
| Proximal point | $f(y)$ itself | no approximation at all |

Table 1: The three model choices. Going down the table the model becomes more faithful to the real function, and (as the experiments show) more forgiving about the step-size.

The nice surprise is that for phase retrieval and blind deconvolution, the minimization in (2) can be solved *exactly* with a short formula for all three methods - we never have to run an inner optimizer. When no formula is available, our code falls back to an inner L-BFGS solve.

### 2.3 The Moreau envelope

For a nonconvex, nonsmooth function we cannot just look at the gradient and we usually do not know the optimal value $\varphi^\star$, so we cannot look at the gap either. Authors of the paper propose the **Moreau envelope**:

$$\varphi_\lambda(x) = \min_y \Big\{\varphi(y) + \tfrac{1}{2\lambda}\|y-x\|^2\Big\}, \qquad \lambda < \tfrac{1}{\rho}. \tag{3}$$

This is a smoothed version of $\varphi$. Even though $\varphi$ is not smooth, $\varphi_\lambda$ *is*, and its gradient has a simple form,

$$\nabla\varphi_\lambda(x) = \tfrac{1}{\lambda}\big(x - \operatorname{prox}_{\lambda\varphi}(x)\big). \tag{4}$$

The Moreau envelope property: if $\|\nabla\varphi_\lambda(x)\|$ is small, then $x$ is close to a point that is nearly stationary for the real $\varphi$ - and we can check this *without knowing* $\varphi^\star$. In our project we use $\lambda = 1/(2\rho)$.

The paper's headline guarantee, stated loosely, is:

> If you run the method with a step-size tuned to the budget ($\beta \propto \sqrt{T}$), then the best point you have seen satisfies $\min_{t\le T}\|\nabla\varphi_\lambda(x_t)\| = O(T^{-1/4})$.

## 3 The main algorithm

### 3.1 The generic solver

The core of our project is one universal solver that we use in each experiment. The only problem-specific call is `solve_exact`, which either returns the closed-form solution of (2) or returns nothing (in which case we use L-BFGS). In pseudocode:

> **given** problem, data $\{\xi_i\}$, start $x_0$, horizon $T$, step-size $\beta$
> **for** $t = 0, 1, \dots, T-1$:
>  $\beta_t \leftarrow \beta(t)$ if $\beta$ is a schedule, else $\beta$
>  draw a random mini batch $\Xi_t$ from the data
>  $x_{\text{exact}} \leftarrow$ `problem.solve_exact`$(x_t, \Xi_t, \beta_t)$
>  **if** a closed form exists: $x_{t+1} \leftarrow x_{\text{exact}}$
>  **else**: minimize $f_{x_t}(y) + r(y) + \frac{\beta_t}{2}\|y - x_t\|^2$ with L-BFGS
>  every few steps: log $\varphi(x_{t+1})$, $\|x_{t+1}\|$, and (optionally) $\|\nabla\varphi_\lambda(x_{t+1})\|$
>  optionally stop early once the progress measure stays below a tolerance
> **return** $x_T$

This is `ModelBasedSolver.run` in `src/ModelBasedSolver.py`. The parameters are: `beta` (a number or a function $\beta(t)$), `batch_size` (we use 1, one sample per step, like the paper), `moreau_every` (how often to compute the expensive progress measure), and `stop_measure` (the built-in early stop, used in Experiment 5.5).

### 3.2 The three update formulas

For phase retrieval, $f(x) = |\langle a, x\rangle^2 - b|$, each step reduces to a move along the direction $a$. Writing $\lambda = 1/\beta$:

**Subgradient.** Take one subgradient and step against it:

$$x_{t+1} = x_t - \lambda\, v, \qquad v = 2\langle a, x_t\rangle\, \operatorname{sign}\big(\langle a, x_t\rangle^2 - b\big)\, a.$$

This is the cheapest and the most fragile method.

**Prox-linear.** Linearize the inside of the absolute value but keep the kink. This gives a step that is automatically "clipped" (paper eq. 5.2):

$$x_{t+1} = x_t + \frac{1}{\beta}\, \operatorname{proj}_{[-1,1]}\Big(-\frac{\gamma\beta}{\|\zeta\|^2}\Big)\, \zeta, \qquad \gamma = \langle a, x_t\rangle^2 - b, \quad \zeta = 2\langle a, x_t\rangle\, a.$$

The projection onto $[-1, 1]$ is what stops it from overshooting, and is the reason it tolerates large step-sizes.

**Proximal point.** Keep the function in its exact form. The subproblem $\arg\min_y |\langle a, y\rangle^2 - b| + \frac{\beta}{2}\|y - x_t\|^2$ has at most four candidate solutions (paper eq. 5.4):

$$y = x_t - \frac{2\lambda\langle a, x_t\rangle}{2\lambda\|a\|^2 \pm 1}\, a, \qquad y = x_t - \frac{\langle a, x_t\rangle \mp \sqrt{b}}{\|a\|^2}\, a,$$

and we simply evaluate all of them and keep the best. For blind deconvolution $f(x, y) = |\langle u, x\rangle\langle v, y\rangle - b|$ the same three ideas apply; the proximal-point case there needs us to solve a small quartic equation to find the candidates (paper eqs. 5.6–5.8).

### 3.3 Computing the progress measure

To get $\|\nabla\varphi_\lambda(x)\|$ we need $\operatorname{prox}_{\lambda\varphi}(x)$, which we find with a full-batch inner L-BFGS solve, and then plug into (4). This lives in `WeaklyConvexProblem.compute_moreau_grad_norm` and is what we test in Experiments 5.4, 5.6, and 5.5. It is relatively resource-expensive, so we only compute it every few epochs.

## 4 Code structure

| File | What it does |
|---|---|
| `src/ModelBasedSolver.py` | the generic loop, logging, and early stopping |
| `src/WeaklyConvexProblem.py` | base problem $\varphi = f + r$, Moreau measure, `solve_exact` hook |
| `src/problems/phase_retrieval.py` | the three closed-form methods for phase retrieval |
| `src/problems/blind_deconvolution.py` | the three closed-form methods for blind deconvolution |
| `src/problems/robust_regression.py` | subgradient on a robust $\ell_1$ regression loss |
| `src/problems/sparse_phase_retrieval.py` | phase retrieval with an $\ell_1$ regularizer |
| `src/regularizers.py` | $\ell_1$ / elastic-net penalties and the soft-threshold prox |
| `src/utils.py` | sign-invariant distance and spectral initialization |
| `experiments/` | one standalone script per experiment (all parallelized) |
| `deliverables/figures/` | the generated figures shown below |

Table 2: Where things live in the repository.

Two design choices are worth mentioning:

- **One solver, many problems.** Every problem is a subclass of `WeaklyConvexProblem` that only has to implement `solve_exact`. This mirrors the paper, where the same proximal update (2) is used and only the model changes.
- **The sign ambiguity.** Phase retrieval can only recover $\bar{x}$ up to a global sign (because $x$ and $-x$ give the same $b$), so we measure recovery with $\text{dist}_{\pm}(x, \bar{x}) = \min(\|x - \bar{x}\|, \|x + \bar{x}\|)$ in `src/utils.py`.

# 5 Experiments

**Shared setup.** Unless stated otherwise: measurement vectors $a_i \sim \mathcal{N}(0, I_d)$, the target $\bar{x}$ on the unit sphere, one sample per step (one *epoch* $= m$ steps), and results averaged over several random rounds. Step-sizes are either a constant $\beta$ swept over $\beta^{-1} \in [10^{-4}, 1]$, or a schedule $\beta_t \propto \sqrt{t}$ for the rate experiment. All figures are produced by the scripts in `experiments/` and saved to `deliverables/figures/`.

## 5.1 Reproduction - Phase Retrieval

This is the reproduction of Figure 3 from the paper. For each step-size we record the function gap after 100 epochs (left panel) and how many epochs it took to reach a small target accuracy (right panel), averaged over 15 rounds. Similarly to the authors of paper we run three problem sizes so we can see whether the picture holds as the dimension grows.

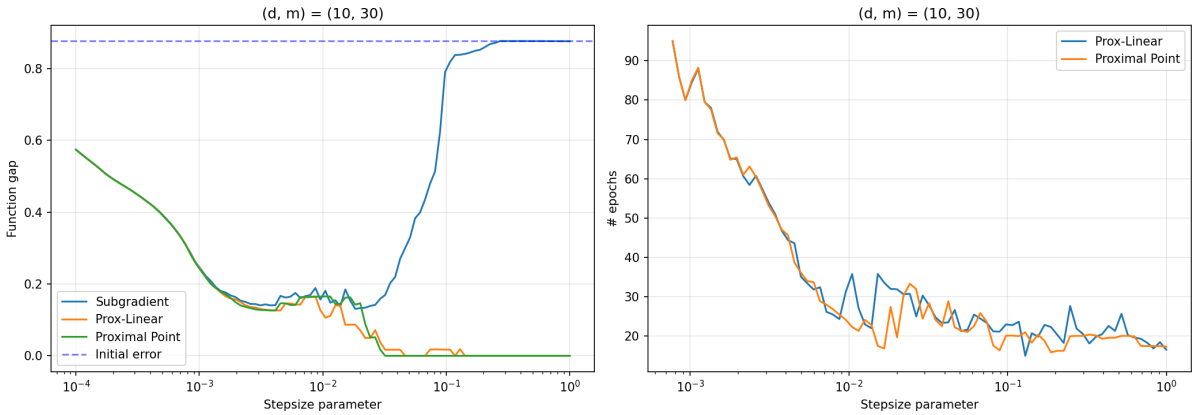


Figure 1: Phase retrieval: $(d, m) = (10, 30)$

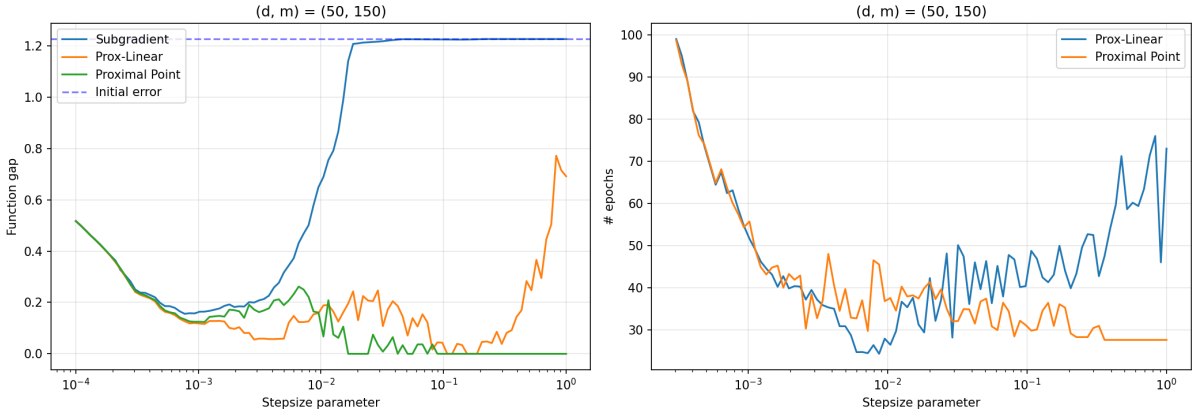


Figure 2: Phase retrieval: $(d, m) = (50, 150)$

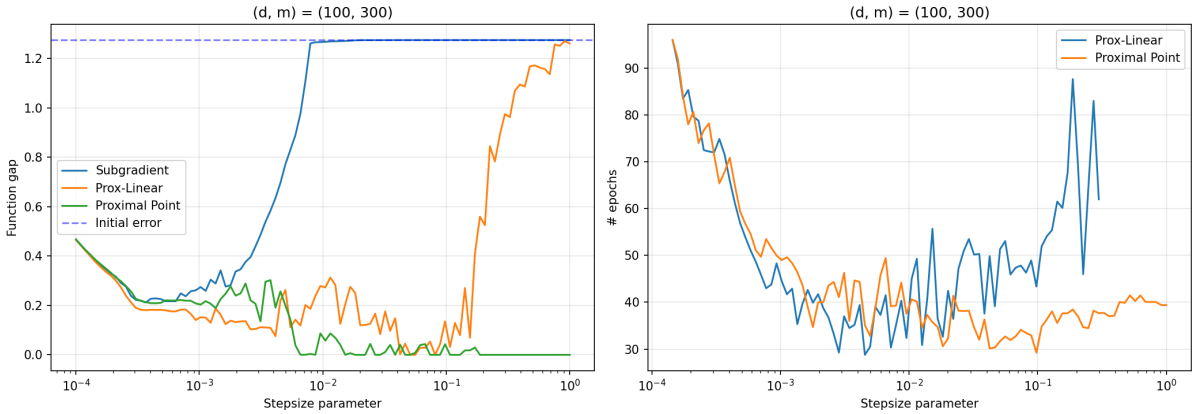


Figure 3: Phase retrieval: $(d, m) = (100, 300)$

**Comments.** We managed to get the similarly shaped curves to the one in paper. In fact we got identical, except they used a uniform distribution for beta on $[10^{-4}, 10^{0}]$ , while we used a log-uniform distribution over the same interval. This process wasn't as straightforward as it seams, the authors didn't took the function gap after 100 epochs as stated, but they took the best one. Otherwise the subgradient blew-up to $10^{23}$.

All three sizes are similar, which is what we expected from the paper. Look at the left panel of any of these figures: the subgradient curve (blue) is a narrow "bathtub" - it only reaches a low function gap in a thin band of step-sizes (around $\beta^{-1} \in [10^{-3}, 10^{-2}]$) and otherwise sits at the initial error, meaning it made no progress at all. The prox-linear (orange) and proximal-point (green) curves converges for a wide range of beta. In other words, the model-based methods are more forgiving: you can be sloppy about the step-size and still converge. The right panel shows that when both model-based methods do converge, they need a comparable number of epochs, with proximal point staying a bit steadier at the largest step-sizes. The picture is essentially unchanged from $d = 10$ up to $d = 100$, which is reassuring - the advantage is not an artifact of a tiny problem.

## 5.2 Reproduction - Blind Deconvolution

To check that the story is not special to phase retrieval, we repeat it on a second, harder problem: blind deconvolution, $\min_{x,y} \frac{1}{m} \sum_i |\langle u_i, x\rangle\langle v_i, y\rangle - b_i|$. Here we are recovering *two* unknown signals at once, so the loss is bilinear rather than quadratic. This reproduces the paper's Figure 4.

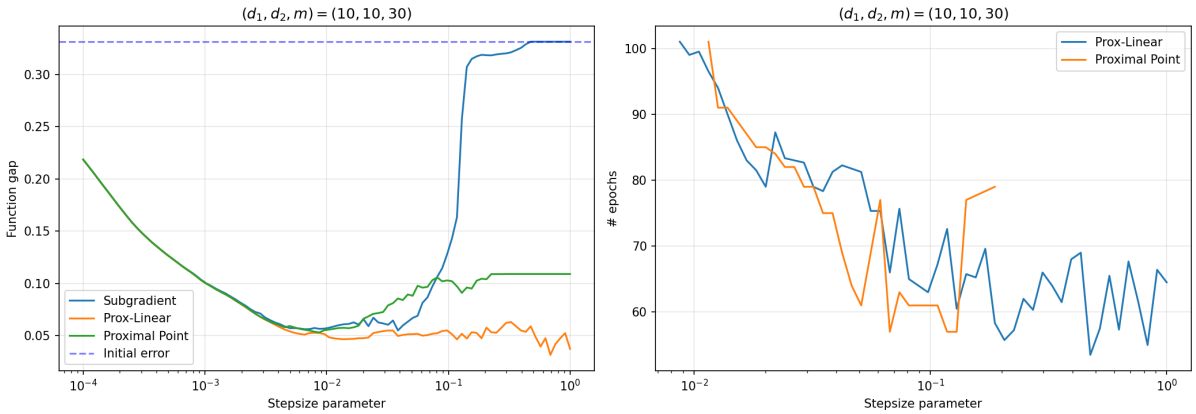


Figure 4: Blind deconvolution: $(d_1, d_2, m) = (10, 10, 30)$

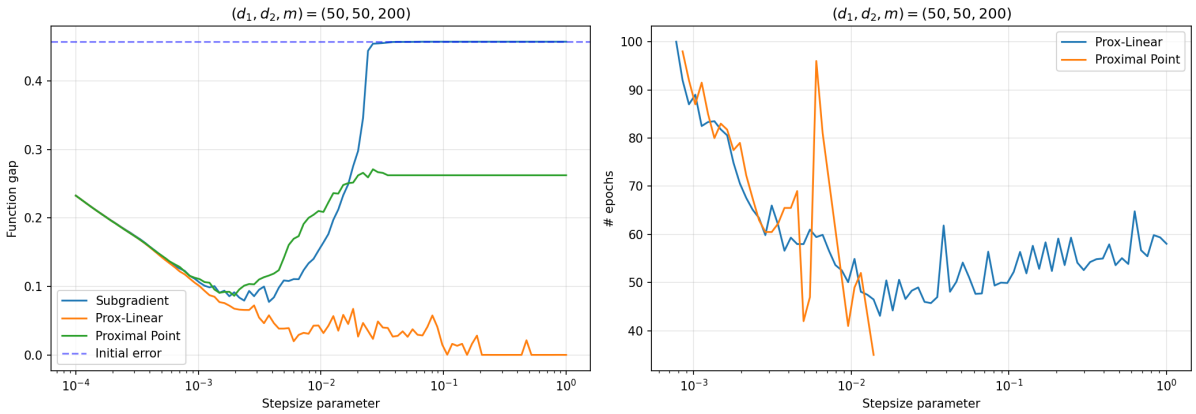


Figure 5: Blind deconvolution: $(d_1, d_2, m) = (50, 50, 200)$

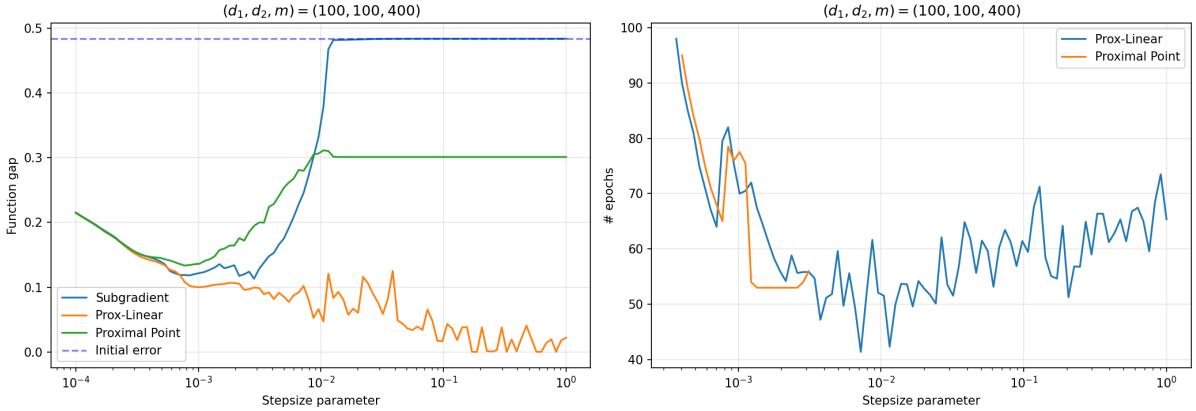


Figure 6: Blind deconvolution: $(d_1, d_2, m) = (100, 100, 400)$

**Comments.** Here the **prox-linear** is the strongest method - its function gap drops essentially to zero over a wide step-size range - while **proximal point** plateaus at a moderate gap once the step-size gets large. We think this is because the bilinear subproblem is genuinely harder: the proximal-point update has to pick among candidates coming from a quartic equation, and that selection is more sensitive to the step-size than the simple clipped step of prox-linear. The subgradient method again collapses to the initial error past a certain step-size. The behaviour is consistent across all three problem sizes. This results differ, from what we seen in the paper, we are not sure why, on the right side plot we have the orange line drawn to some point around $10^{-1}$ later it did not converged, so we cannot say after how may epochs it crossed the threshold. Also it takes longer (30 vs 60) for prox linear to converged compared to Figure 4 in the paper that deals with the same problem. Probably the devil is in the details and we made some other assumption than the authors.

### 5.3 Comparison with Gradient-Based Optimizers

It is natural to ask: are the model-based methods actually better than the optimizers everyone already uses? Here we put the three model-based methods up against SGD, Adam, and AdaGrad on the exact same phase-retrieval loss (the standard optimizers use autograd on the loss), sweeping the learning rate.

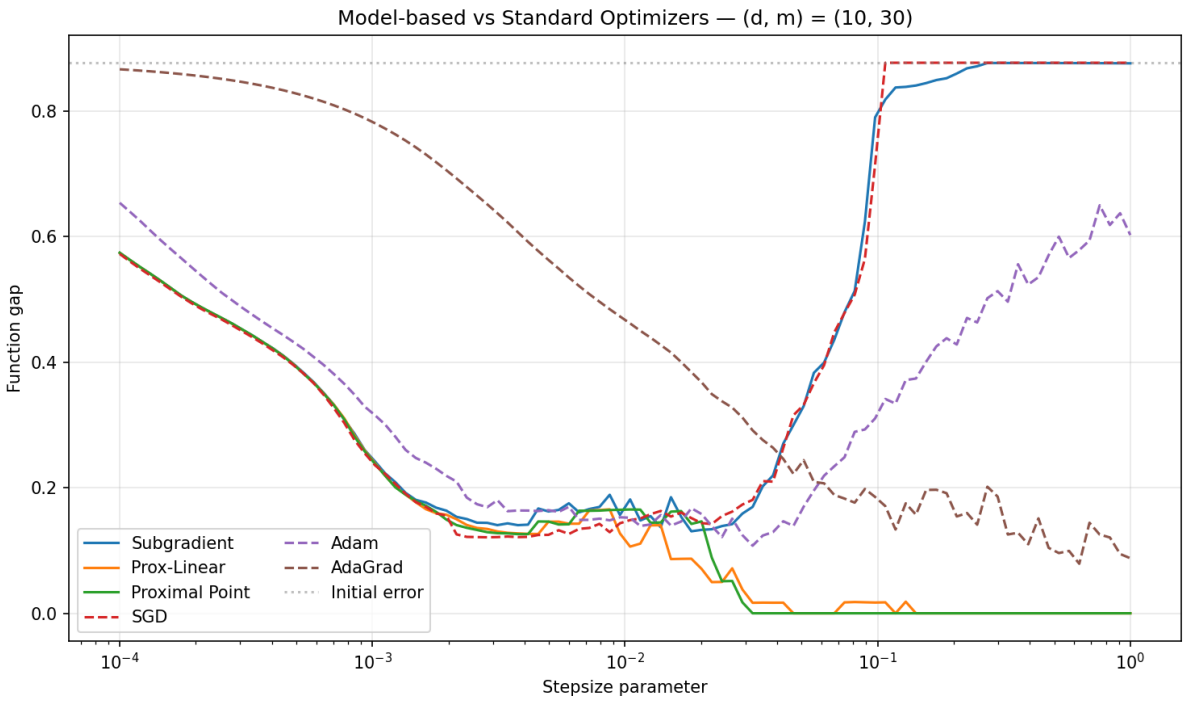


Figure 7: Optimizer comparison: $(d, m) = (10, 30)$

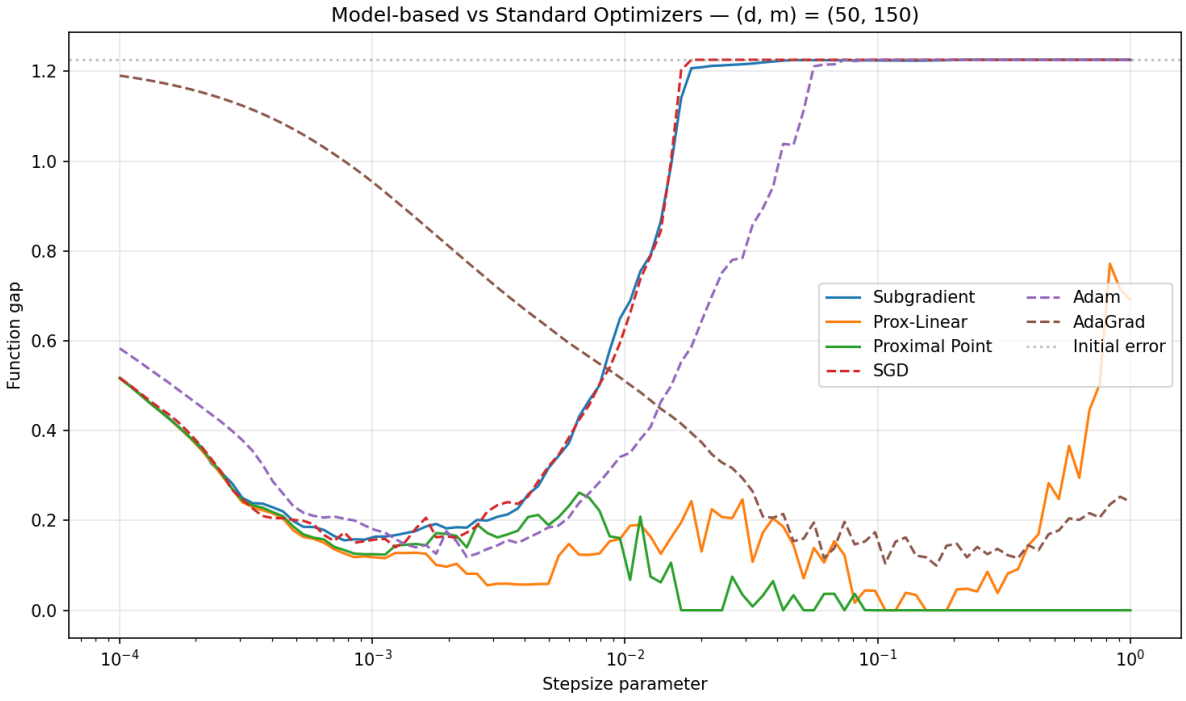


Figure 8: Optimizer comparison: $(d, m) = (50, 150)$

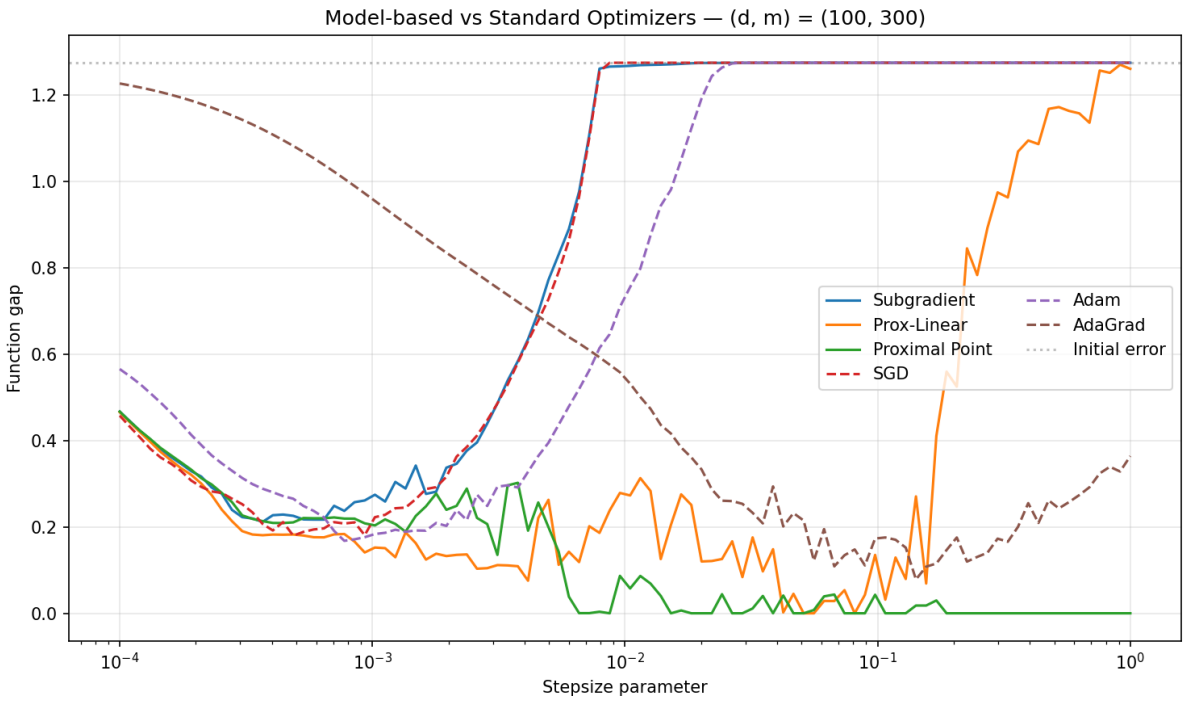


Figure 9: Optimizer comparison: $(d, m) = (100, 300)$

**Comments.** The proximal-point method (green) is the only optimizer that reaches a function gap of (almost) zero over a wide range of large step-sizes. SGD (red dashed) behaves just like the plain subgradient method and falls apart once the step-size is too big. Adam and AdaGrad do better than SGD - their adaptive scaling widens the usable band - but they still degrade outside a tuned region and never quite match proximal point's "set it large and forget it" behaviour. The lesson we take away is that the robustness comes from solving a proximal *subproblem* with a faithful model, not from clever gradient rescaling. This holds across all three sizes.

### 5.4 Moreau Envelope Gradient Norm

The previous plots used the function gap, which we can only compute because we happen to know the answer. This experiment tracks the paper's real progress measure, $\|\nabla\varphi_\lambda(x_t)\|$, over training using a **decreasing step-size schedule**

$$\beta_t = \rho + \frac{\beta_0 - \rho}{\sqrt{1 + t/m}},$$

which decays at $O(1/\sqrt{\text{epoch}})$ while maintaining $\beta_t > \rho$. Each method uses a different initial $\beta_0$ scaled by a per-method multiplier (Subgradient: $20\times$, Prox-Linear: $1\times$, Proximal Point: $0.5\times$) to account for model accuracy differences—the rougher the model, the stronger the initial regularization required. The base $\beta_0^{-1} = 0.01$. The left panel of each figure is the Moreau gradient norm; the right panel is the actual objective value on a log scale. The left panel of each figure is the Moreau gradient norm; the right panel is the actual objective value on a log scale.

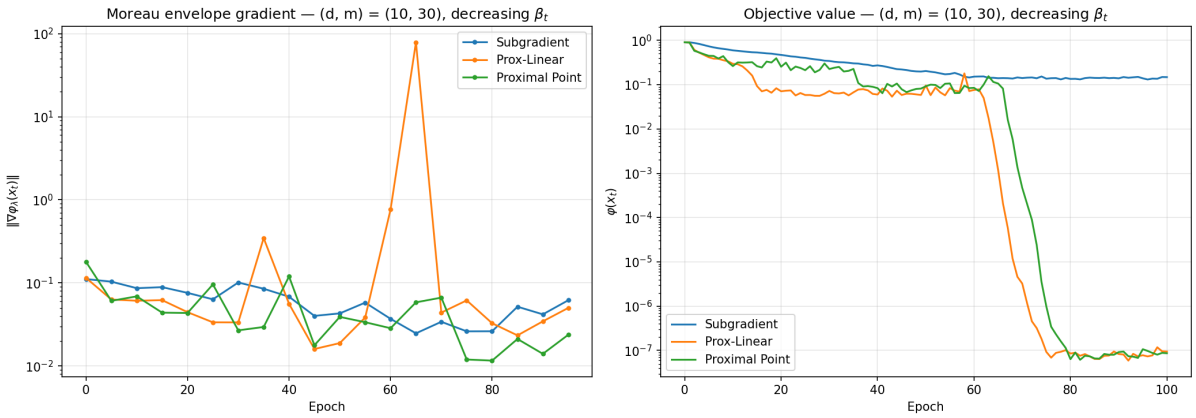


Figure 10: Moreau envelope gradient norm: $(d, m) = (10, 30)$

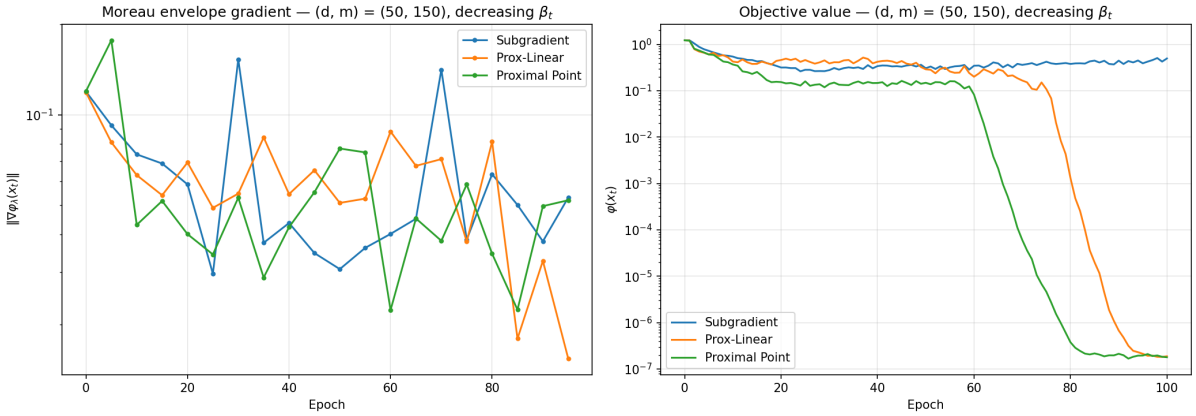


Figure 11: Moreau envelope gradient norm: $(d, m) = (50, 150)$

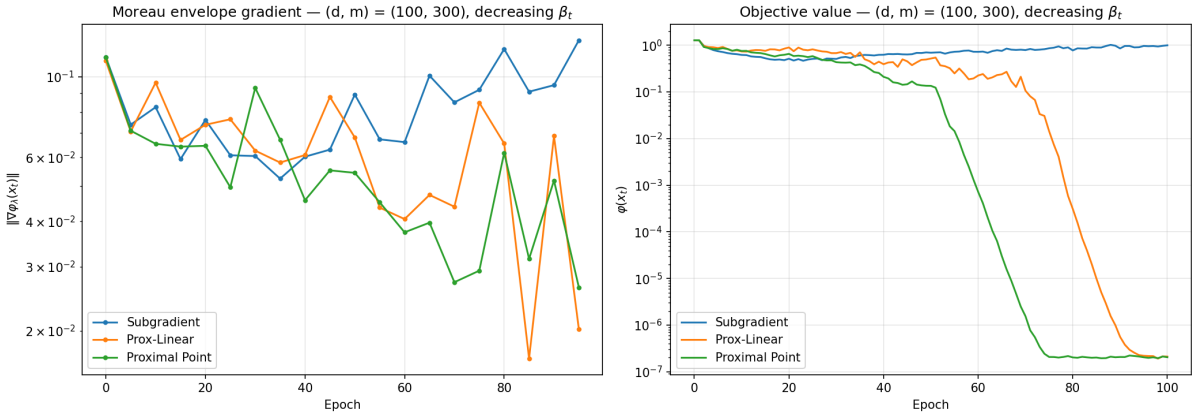


Figure 12: Moreau envelope gradient norm: $(d, m) = (100, 300)$

**Comments.** The two panels move together: when the Moreau gradient norm goes down, the true objective goes down too. That is exactly what we wanted to check - it means the Moreau measure is a trustworthy progress signal that we could use even on a problem where we do not know the answer. Proximal point (green) descends the fastest, pushing the objective all the way down to about $10^{-7}$. Very similar is prox-linear (orange) but it stays in a platou regions for

longer. Both algorithms have a characteristic "long plateau then a sudden cliff": they lingers near a flat region before locking onto the basin of attraction and dropping sharply. The subgradient method (blue) also decreases but to the point, then it stops and stays at around $10^{-1}$. Norm of Moreau Envelope gradient, for proximal point and prox-linear decreases to around $10^{-2}$.

### 5.5 Stopping Criteria

A practical question the theory lets us answer: *when should we stop?* Because the Moreau gradient norm does not need to know $\varphi^\star$, it can act as a real stopping certificate. Here we compare four stopping rules on a proximal-point run: (1) Moreau gradient norm below a threshold, (2) the function gap below a threshold (an "oracle" rule, only possible because here $\varphi^\star = 0$), (3) iterate stagnation $\|x_{t+1} - x_t\|$ small, and (4) just running to a fixed budget.

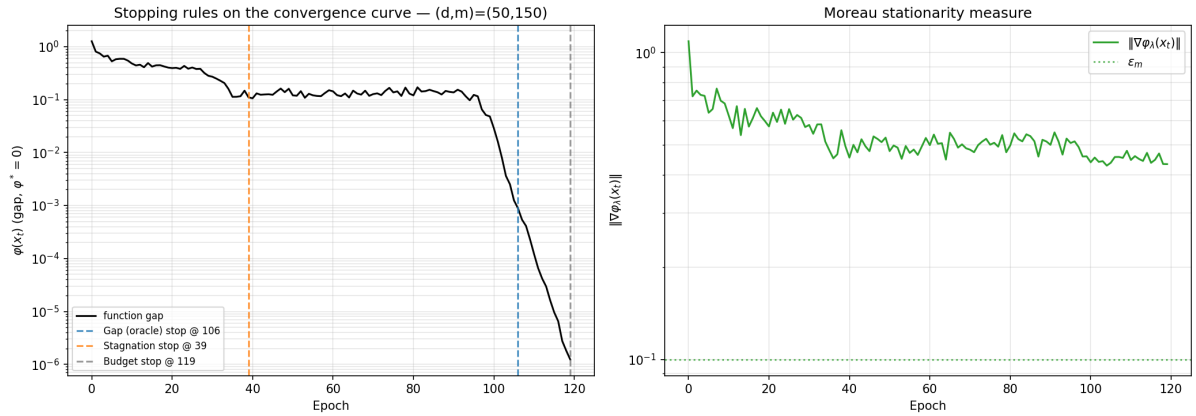


Figure 13: Comparison of four stopping rules on phase retrieval with $(d, m) = (50, 150)$.

**Comments.** This figure is a nice illustration of why the naive stopping rule is dangerous. The function-gap curve (left) sits on a long plateau near $10^{-1}$ and only drops off a cliff after about epoch 95. The cheap **stagnation** rule fires way too early (around epoch 39): it sees the flat plateau and wrongly concludes we have converged, stopping before any real progress happens. The **oracle gap** rule (around epoch 106) is the gold standard but cheats, since it needs to know the optimum. The **Moreau** gradient norm (right panel) should help us to decide when to stop, but as we can see in this example the Moreau gradient norm never drops and stays at around 0.7. We tried to run it for 1000 epochs but this fact doesn't changes. So in this example the Moreau envelope gradient norm cannot be used as a stopping criteria. That is not in line with what theory promised, we are not sure why we got such results. The fixed-budget rule (epoch 119) is safer but leads to wasted computation.

### 5.6 Convergence Rate Experimentally

The earlier experiments show the progress measure goes to zero. This experiment tests stronger, quantitative claim: that it does so at the predicted $O(k^{-1/4})$ rate. We use two views. *Left (A1):* a single long run with a small constant step-size, plotting the running minimum $\min_{t \le k} \|\nabla \varphi_\lambda(x_t)\|$ against iteration on a log-log scale, next to a reference line of slope $-1/4$. *Right (A2, the rigorous test):* for several time budgets $T$ we set $\beta \propto \sqrt{T}$ as the theory requires, record the best stationarity reached, and fit the slope.

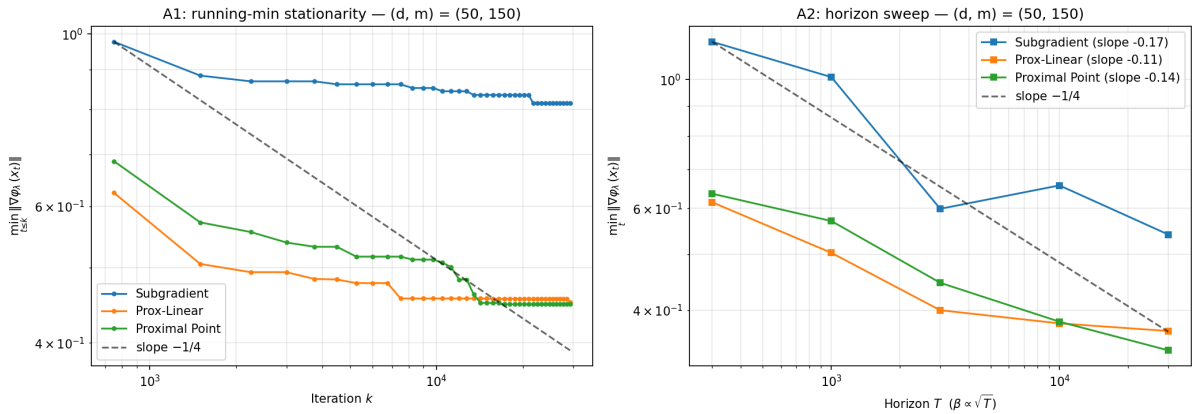


Figure 14: Empirical verification of the $O(k^{-1/4})$ convergence rate with $(d, m) = (50, 150)$. Left: running-minimum stationarity over a single long run. Right: best stationarity vs. horizon $T$ with $\beta \propto \sqrt{T}$.

**Comments.** The left panel behaves as expected: the running minimum decays along the $-1/4$ reference for a while and then flattens out at a "noise floor" - with a fixed small step-size, the randomness in the sampling eventually dominates and you cannot get below a certain level. On the right, the fitted slopes ($-0.17$ for subgradient, $-0.11$ for prox-linear, $-0.14$ for proximal point) are the right sign and the right ballpark but *shallower* than the theoretical $-1/4$. We read this as a confirmation rather than a contradiction, for two honest reasons: (i) $-1/4$ is a worst-case upper bound, and on these well-behaved Gaussian problems the methods reach their noise floor before the asymptotic regime kicks in; and (ii) the model-based methods start from a much lower error, so their curves are already near the floor across the whole range, which flattens the measured slope. The practical takeaway matches all the earlier plots: what separates the methods is the constant, not the exponent.

### 5.7 Empirical Recovery Probability

Finally, a statistical question: how many measurements do we actually need to recover the signal? We fix $d = 50$ and sweep the oversampling ratio $m/d \in \{1, \ldots, 6\}$. For each ratio we run 50 independent trials (fresh data and random start) and count a trial as a success if the sign-invariant recovery error drops below $10^{-3}$.

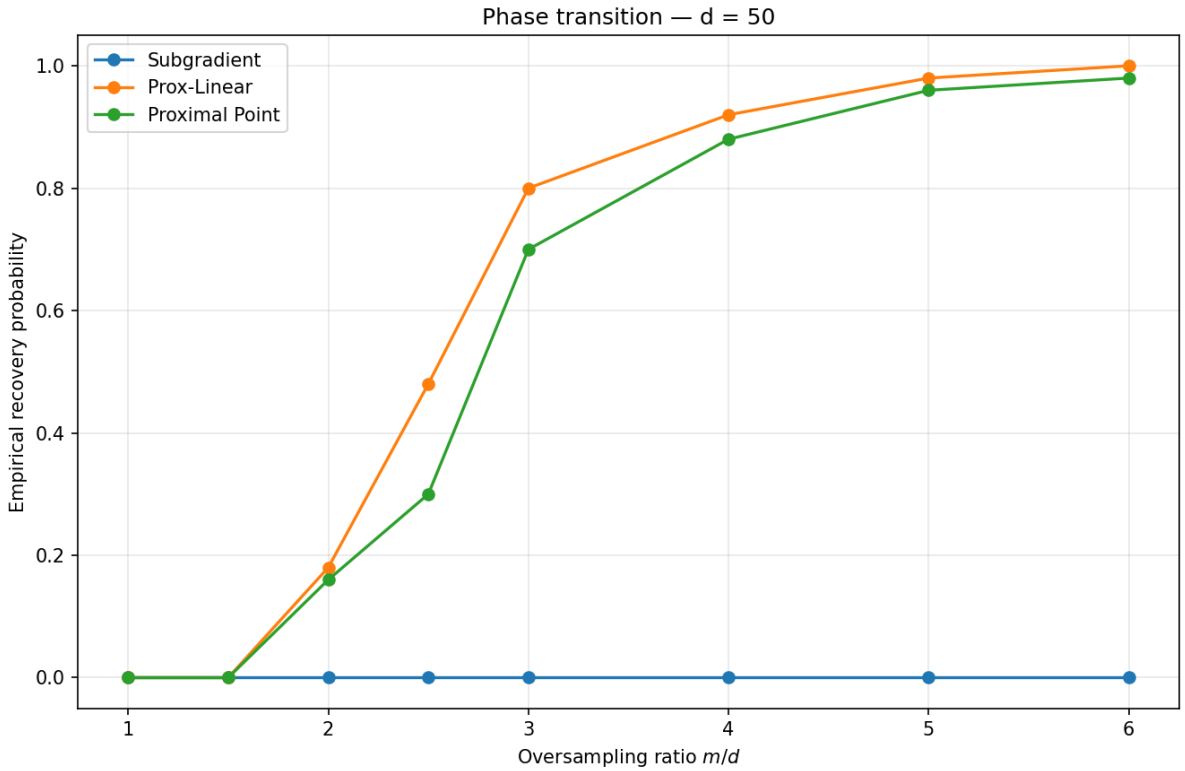


Figure 15: Recovery probability vs. oversampling ratio $m/d$ for the three model-based methods, $d = 50$, 50 trials per ratio.

**Comments.** There is a clean **phase transition**: below about $m/d \approx 1.5$ recovery is essentially impossible (not enough measurements), and above about $m/d \approx 5$ it is almost guaranteed, with a sharp S-shaped rise in between. Prox-linear (orange) crosses over slightly earlier than proximal point (green). The subgradient line (blue) stays flat at zero. Under the fixed number of epochs and the untuned random step-size used here, the subgradient method simply does not reach the strict $10^{-3}$ tolerance - which is completely consistent with its narrow usable step-size band from the very first experiment.

# 6 Summary

Synthesizing the experimental results across phase retrieval and blind deconvolution tasks, the project validates the theoretical guarantees of the Davis and Drusvyatskiy (2019) framework. The core findings are as follows:

- **Robustness:** The prox-linear and proximal point methods demonstrate stability across a broad spectrum of step-sizes (spanning approximately two/three orders of magnitude). In contrast, the subgradient method and standard gradient-based optimizers (such as SGD) exhibit high sensitivity and require rigorous step-size tuning to achieve convergence.
- **Moreau Envelope:** The Moreau envelope seems like a reliable, computable approach optimization problems. It accurately tracks convergence progress without requiring prior knowledge of the optimal function value.
- **Stopping Criteria:** Relying on iterate stagnation often triggers premature termination on flat loss regions. Moreover, using the Moreau gradient norm not always provides an

effective stopping mechanism that closely approximates the ideal "oracle" stopping point, as we showed in one example the Moreau gradient norm not always converges.

- **Empirical Convergence Rates:** The measured best-iterate stationarity exhibits polynomial decay. While the empirically fitted slopes are somewhat shallower than the theoretical worst-case bound of $O(k^{-1/4})$ - primarily due to rapid convergence to the noise floor and low initial errors-the fundamental convergence behavior aligns with the theoretical predictions.

**Reproducing the figures.** Each experiment is a standalone script in `experiments/` and every figure above is saved to `deliverables/figures/`.

## References

[1] D. Davis and D. Drusvyatskiy. Stochastic model-based minimization of weakly convex functions. *SIAM Journal on Optimization*, 29(1):207–239, 2019.